

Scriptingverslag

Linux & Windows Server [OBI40a]

Naam: Salaheddine Sennouni

Studentnummer: R1050251

Klasgroep: B TI/2I4

Docent: Maarten Troost

Vak: Linux & Windows Server

Bronnen

Voor het uitwerken van deze opdracht werd gebruik gemaakt van de volgende bronnen:

- Claude AI (Anthropic) — <https://claude.ai>

Vraagstelling: stap-voor-stap begeleiding bij het schrijven van het Bash-script, uitleg van requirements, debugging van syntax fouten en evaluatie van scriptlogica.

Antwoord: per requirement werd de code uitgelegd, getest en verbeterd. Screenshots van de chatgesprekken zijn opgenomen in de sectie AI-gebruik.

- ChatGPT (OpenAI) — <https://chat.openai.com>

Gebruikt voor peer review van het script. Feedback resulteerde in verbeteringen zoals `--exclude` bij `tar`, bredere `grep` regex en verwijderen van onnodige echo bij `req 3+4`.

- GNU Bash Reference Manual — <https://www.gnu.org/software/bash/manual/>
- ShellCheck wiki — <https://www.shellcheck.net/wiki/SC2034>
- Linux man pages: `tar`, `grep`, `chmod`, `mkdir`, `date`

Requirement 1 & 2 — Argumentencheck

Wat wordt getest?

Het script verwacht exact 2 argumenten. Bij 0, 1 of 3+ argumenten stopt het script met de exacte foutmelding en exit code 1:

```
syntax error: huistaascripting.sh werkdirectory aantal
```

Code

```
GNU nano 7.2 huistaascripting.sh
#!/bin/bash

# Req 1+2: controleer of er precies 2 argumenten zijn
if [ "$#" -ne 2 ]; then
    echo "syntax error: huistaascripting.sh werkdirectory aantal" >&2
    exit 1
fi

WERKDIR="$1"
# $2 (aantal) wordt als argument vereist maar verder niet gebruikt in dit script
```

Nano: req 1+2 — argumentencheck

Testresultaat

Geen argumenten en 1 argument: foutmelding + exit code 1.

```
salah@LenovoStudent:~$ ./huistaascripting.sh
syntax error: huistaascripting.sh werkdirectory aantal
salah@LenovoStudent:~$ ./huistaascripting.sh /tmp
syntax error: huistaascripting.sh werkdirectory aantal
```

Test: geen args en 1 arg geven foutmelding

3 argumenten: ook foutmelding + exit code 1.

```
salah@LenovoStudent:~$ ./huistaascripting.sh ~/werktest 3 extra
echo $?
syntax error: huistaascripting.sh werkdirectory aantal
1
```

Test: 3 argumenten geeft ook foutmelding + exit code 1

Requirement 3 & 4 — Werkdiratory check

Wat wordt getest?

Alle oorzaken waarom een werkmap niet bruikbaar is, gecombineerd in 1 conditie. Getest voor: map bestaat niet, pad is een bestand, geen leesrecht, geen schrijfrecht, geen uitvoerrecht. Het script faalt zonder foutboodschap (enkel exit code 1).

Code

```
# Req 3+4: alle werkdir-checks gecombineerd in 1 conditie
if [ ! -d "$WERKDIR" ] || [ ! -r "$WERKDIR" ] || [ ! -w "$WERKDIR" ] || [ ! -x "$WERKDIR" ]
then
    exit 1
fi
```

Nano: req 3+4 — werkdircheck in 1 conditie

Testresultaat — alle gevallen

Alle ongeldige gevallen geven exit code 1 zonder foutmelding. Geldige map geeft exit code 0.

```
salah@LenovoStudent:~$ ./huistaascripting.sh ~/bestaatNiet 3
echo $?
1
salah@LenovoStudent:~$ touch ~/geenmap.txt
./huistaascripting.sh ~/geenmap.txt 3
echo $?
1
salah@LenovoStudent:~$ mkdir -p ~/test_no_read
chmod u-r ~/test_no_read
./huistaascripting.sh ~/test_no_read 3
echo $?
1
salah@LenovoStudent:~$ mkdir -p ~/test_no_write
chmod u-w ~/test_no_write
./huistaascripting.sh ~/test_no_write 3
echo $?
1
salah@LenovoStudent:~$ mkdir -p ~/test_no_exec
chmod u-x ~/test_no_exec
./huistaascripting.sh ~/test_no_exec 3
echo $?
1
salah@LenovoStudent:~$
```

Test req 3+4: bestaatNiet, bestand, geen-r, geen-w, geen-x — exit 1

```
salah@LenovoStudent:~$ ./huistaascripting.sh ~/bestaatNiet 3
echo "Exit code was: $?"
Exit code was: 1
salah@LenovoStudent:~$
```

Test req 3+4: geen foutmelding, enkel exit code 1

Requirement 5 — copy_bashrc aanmaken

Wat wordt getest?

Het script kopieert ~/.bashrc naar de werkmap als copy_bashrc en voegt de functie clear_history() toe die de bash-history wist.

Code

```
# Req 3+4: alle werkdir-checks gecombineerd in 1 conditie
if [ ! -d "$WERKDIR" ] || [ ! -r "$WERKDIR" ] || [ ! -w "$WERKDIR" ] || [ ! -x "$W
```

Nano: req 5 — cp HOME/.bashrc

Testresultaat — bestand aangemaakt met functie

Na uitvoering is copy_bashrc aanwezig met de originele bashrc-inhoud en clear_history() onderaan.

```
salah@LenovoStudent:~$ ./huistaakscripting.sh ~/werktest 3
salah@LenovoStudent:~$ cat ~/werktest/copy_bashrc
# ~/.bashrc: executed by bash(1) for non-login shells.
# see /usr/share/doc/bash/examples/startup-files (in the package bash-doc)
# for examples

# If not running interactively, don't do anything
case $- in
    *) ;;
    *) return;;
esac

# don't put duplicate lines or lines starting with space in the history.
# See bash(1) for more options
HISTCONTROL=ignoreboth

# append to the history file, don't overwrite it
shopt -s histappend

# for setting history length see HISTSIZE and HISTFILESIZE in bash(1)
HISTSIZE=1000
HISTFILESIZE=2000

# check the window size after each command and, if necessary,
# update the values of LINES and COLUMNS.
shopt -s checkwinsize

# If set, the pattern "*" used in a pathname expansion context will
# match all files and zero or more directories and subdirectories.
#shopt -s globstar

# make less more friendly for non-text input files, see lesspipe(1)
[ -x /usr/bin/lesspipe ] && eval "$(SHELL=/bin/sh lesspipe)"

# set variable identifying the chroot you work in (used in the prompt below)
if [ -z "${debian_chroot:-}" ] && [ -r /etc/debian_chroot ]; then
    debian_chroot=$(cat /etc/debian_chroot)
fi

# set a fancy prompt (non-color, unless we know we "want" color)
case "$TERM" in
    xterm-color|*-256color) color_prompt=yes;;
esac

# uncomment for a colored prompt, if the terminal has the capability; turned
# off by default to not distract the user: the focus in a terminal window
# should be on the output of commands, not on the prompt
#force_color_prompt=yes

if [ -n "$force_color_prompt" ]; then
    if [ -x /usr/bin/tput ] && tput setaf 1 >&/dev/null; then
        # We have color support; assume it's compliant with Ecma-48
        # (ISO/IEC-6429). (Lack of such support is extremely rare, and such
        # a case would tend to support setf rather than setaf.)
        color_prompt=yes
    else
        color_prompt=
    fi
fi

if [ "$color_prompt" = yes ]; then
    PS1='${debian_chroot:+($debian_chroot)}\[\033[01;32m\]\u@\h\[\033[00m\]:\[\033[01;34m\]\w\[\033[00m\]\$ '
else
    PS1='${debian_chroot:+($debian_chroot)}\u@\h:\w$ '
fi
unset color_prompt force_color_prompt
```

Inhoud copy_bashrc deel 1

```
unset color_prompt force_color_prompt

# If this is an xterm set the title to user@host:dir
case "$TERM" in
xterm*|rxvt*)
    PS1="\[\e]0;${debian_chroot:+($debian_chroot)}\u@\h: \w\a\]$PS1"
    ;;
*)
    ;;
esac

# enable color support of ls and also add handy aliases
if [ -x /usr/bin/dircolors ]; then
    test -r ~/.dircolors && eval "$(dircolors -b ~/.dircolors)" || eval "$(dircolors -b)"
    alias ls='ls --color=auto'
    #alias dir='dir --color=auto'
    #alias vdir='vdir --color=auto'

    alias grep='grep --color=auto'
    alias fgrep='fgrep --color=auto'
    alias egrep='egrep --color=auto'
fi

# colored GCC warnings and errors
#export GCC_COLORS='error=01;31:warning=01;35:note=01;36:caret=01;32:locus=01:quote=01'

# some more ls aliases
alias ll='ls -alF'
alias la='ls -A'
alias l='ls -CF'

# Add an "alert" alias for long running commands. Use like so:
# sleep 10; alert
alias alert='notify-send --urgency=low -i "${[ $? = 0 ] && echo terminal || echo error}" "${history|tail -n1|sed -e '\''s/\s*[0-9]\+\s+//;s/[:&]\s*alert$//'\''}"'

# Alias definitions.
# You may want to put all your additions into a separate file like
# ~/.bash_aliases, instead of adding them here directly.
# See /usr/share/doc/bash-doc/examples in the bash-doc package.

if [ -f ~/.bash_aliases ]; then
    . ~/.bash_aliases
fi

# enable programmable completion features (you don't need to enable
# this, if it's already enabled in /etc/bash.bashrc and /etc/profile
# sources /etc/bash.bashrc).
if ! shopt -oq posix; then
    if [ -f /usr/share/bash-completion/bash_completion ]; then
        . /usr/share/bash-completion/bash_completion
    elif [ -f /etc/bash_completion ]; then
        . /etc/bash_completion
    fi
fi

clear_history() {
    history -c
    history -w
}

salah@LenovoStudent:~$ |
```

Inhoud copy_bashrc deel 2 — clear_history() staat onderaan

Testresultaat — functie werkt

source laadt de functie, clear_history wist de history, history toont enkel nog de laatste regel.

```
salah@LenovoStudent:~$ source ~/werktest/copy_bashrc
salah@LenovoStudent:~$ clear_history
salah@LenovoStudent:~$ history
  1  history
salah@LenovoStudent:~$
```

Test: source + clear_history + history leeg

Requirement 6 — Functie niet dubbel toevoegen

Wat wordt getest?

Bij herhaalde uitvoering mag de functie `clear_history()` slechts 1 keer in `copy_bashrc` staan. De grep-regex vangt ook varianten op zoals `function clear_history` of `clear_history ()`.

Code

```
# Req 6: voeg functie ALLEEN toe als ze nog niet bestaat
if ! grep -Eq '^[[:space:]]*(function[[:space:]]+)?clear_history[[:space:]]*(\(\))$'
    cat >> "$WERKDIR/copy_bashrc" << 'EOF'

clear_history() {
    history -c
    history -w
}
EOF
fi
```

Nano: req 6 — grep -Eq brede regex voor functiecheck

Testresultaat

Script 2x uitgevoerd, grep -c toont 1 — definitie staat slechts 1 keer in het bestand.

```
salah@LenovoStudent:~$ nano huistaascripting.sh
salah@LenovoStudent:~$ ./huistaascripting.sh ~/werktest 3
salah@LenovoStudent:~$ ./huistaascripting.sh ~/werktest 3
salah@LenovoStudent:~$ grep -c "clear_history" ~/werktest/copy_bashrc
1
salah@LenovoStudent:~$ grep "clear_history()" ~/werktest/copy_bashrc
clear_history() {
salah@LenovoStudent:~$
```

Test req 6: grep -c geeft 1, functie staat niet dubbel

Requirement 7 & 8 — Tar en gzip archief

Wat wordt getest?

Alle bestanden in de werkmap worden gebundeld en gecomprimeerd naar `bestanden.tar.gz`. Via `--exclude` wordt het tar-bestand zelf uitgesloten zodat er geen waarschuwing komt.

Code

```
# Req 7+8: zip alle bestanden in werkdir naar bestanden.tar.gz
tar --exclude='./bestanden.tar.gz' -czf "$WERKDIR/bestanden.tar.gz" -C "$WERKDIR" .
```

Nano: req 7+8 — tar met --exclude

Testresultaat — archief aangemaakt

```
salah@LenovoStudent:~$ shellcheck huistaascripting.sh
salah@LenovoStudent:~$ rm -rf ~/werktest && mkdir ~/werktest
salah@LenovoStudent:~$ ./huistaascripting.sh ~/werktest 3
tar: ..: file changed as we read it
salah@LenovoStudent:~$ ls ~/werktest/
bestanden.tar.gz  copy_bashrc  extracted_20260413_000034_1405
```

ls werktest toont `bestanden.tar.gz`

Testresultaat — inhoud archief

`tar -tzf` toont de bestanden in het archief. Geen tar-waarschuwing dankzij `--exclude`.

```
salah@LenovoStudent:~$ tar -tzf ~/werktest/bestanden.tar.gz | head
./
./extracted_20260413_000034_1405/
./extracted_20260413_000034_1405/copy_bashrc
./copy_bashrc
```

tar -tzf toont inhoud van bestanden.tar.gz

Requirement 9 & 10 — Unieke submap aanmaken

Wat wordt getest?

Bij elke uitvoering wordt een nieuwe unieke submap aangemaakt (datum+tijd+process ID) en wordt bestanden.tar.gz daarin uitgepakt. Bestaande mappen worden nooit hergebruikt.

Code

```
# Req 9+10: maak unieke subdir op basis van datum+tijd en pak uit
SUBDIR="$WERKDIR/extracted_$(date +%Y%m%d_%H%M%S)_$$"
mkdir "$SUBDIR"
tar -xzf "$WERKDIR/bestanden.tar.gz" -C "$SUBDIR"
```

Nano: req 9+10 — unieke subdir via datum+tijd+PID

Testresultaat — 2 unieke subdirs met inhoud

Script 2x uitgevoerd. Twee unieke mappen aangemaakt, elk met uitgepakte bestanden erin.

```
salah@LenovoStudent:~$ ./huistaascripting.sh ~/bestaatNiet 3
salah@LenovoStudent:~$ sleep 2
salah@LenovoStudent:~$ ./huistaascripting.sh ~/werktest 3
salah@LenovoStudent:~$ ls ~/werktest/
bestanden.tar.gz  extracted_20260413_000034_1405
copy_bashrc       extracted_20260413_000154_1421
salah@LenovoStudent:~$
```

ls extracted_ toont 2 unieke mappen*

```
salah@LenovoStudent:~$ ls ~/werktest/extracted_*
/home/salah/werktest/extracted_20260413_000034_1405:
copy_bashrc

/home/salah/werktest/extracted_20260413_000154_1421:
copy_bashrc  extracted_20260413_000034_1405
```

Inhoud van elke extracted_... map zichtbaar

Shellcheck — Linter resultaat

Eerste run (oude script)

Bij de eerste shellcheck-run werd 1 waarschuwing gevonden: SC2034 — de variabele AANTAL werd gedeclareerd maar nooit gebruikt. Opgelost door de variabele te vervangen door commentaar.

Definitieve run — geen errors

Shellcheck geeft geen output en exit code = 0 — volledig schoon script.

```
salah@LenovoStudent:~$ shellcheck huistaascripting.sh && echo "Geen errors gevonden!"
Geen errors gevonden!
```

Shellcheck: geen errors gevonden

```
salah@LenovoStudent:~$ shellcheck huistaascripting.sh
echo $?
0
salah@LenovoStudent:~$ |
```

Shellcheck: exit code 0 = geen fouten

Volledig script

```
salah@LenovoStudent:~$ cat huistaascripting.sh
#!/bin/bash

# Req 1+2: controleer of er precies 2 argumenten zijn
if [ "$#" -ne 2 ]; then
    echo "syntax error: huistaascripting.sh werkdiry aantal" >&2
    exit 1
fi

WERKDIR="$1"
# $2 (aantal) wordt als argument vereist maar verder niet gebruikt in dit script

# Req 3+4: alle werkdir-checks gecombineerd in 1 conditie
if [ ! -d "$WERKDIR" ] || [ ! -r "$WERKDIR" ] || [ ! -w "$WERKDIR" ] || [ ! -x "$WERKDIR" ]; then
    exit 1
fi

# Req 5: kopieer ~/.bashrc naar werkdir als copy_bashrc
cp "$HOME/.bashrc" "$WERKDIR/copy_bashrc"

# Req 6: voeg functie ALLEEN toe als ze nog niet bestaat
if ! grep -Eq '^[:space:]]*(function[:space:]]+)?clear_history[:space:]]*(\(\))?'
[:space:]]*\{' "$WERKDIR/copy_bashrc"; then
    cat >> "$WERKDIR/copy_bashrc" << 'EOF'

clear_history() {
    history -c
    history -w
}
EOF
fi

# Req 7+8: zip alle bestanden in werkdir naar bestanden.tar.gz
tar --exclude='./bestanden.tar.gz' -czf "$WERKDIR/bestanden.tar.gz" -C "$WERKDIR" .

# Req 9+10: maak unieke subdir op basis van datum+tijd en pak uit
SUBDIR="$WERKDIR/extracted_$(date +%Y%m%d_%H%M%S)_$$"
mkdir "$SUBDIR"
tar -xzf "$WERKDIR/bestanden.tar.gz" -C "$SUBDIR"
```

cat huistaascripting.sh — definitieve versie

AI-gebruik — Chatgesprekken met Claude AI

Hieronder staan screenshots van de chatgesprekken met Claude AI (claude.ai) waarbij elk requirement stap voor stap werd uitgelegd en getest.

Gesprek 1 — Uitleg requirement 1 & 2 (argumentencheck)

Het script heeft 2 argumenten nodig: een werkdirectory en een aantal. Als er niet precies 2 argumenten opgegeven worden, moet het script stoppen met een foutmelding.

```
bash

if [ "$#" -ne 2 ]; then
    echo "syntax error: huistaascripting.sh werkdirectory aantal" >&2
    exit 1
fi
```

Uitleg regel per regel:

- `$#` → het aantal argumenten dat meegegeven werd
- `-ne 2` → "not equal to 2", dus niet gelijk aan 2
- `echo "... " >&2` → toont de foutmelding via het foutkanaal
- `exit 1` → stopt het script met foutcode 1

Testen:

```
bash

./huistaascripting.sh           # geen args → foutmelding
./huistaascripting.sh /tmp      # 1 arg → foutmelding
./huistaascripting.sh ~/werktest 3 extra # 3 args → foutmelding
./huistaascripting.sh ~/werktest 3 # 2 args → geen fout
```

Claude AI: uitleg req 1+2 — argumentencheck met code en tests

Gesprek 2 — Uitleg requirement 3 & 4 (werkdirectory check)

De werkdirectory moet gecontroleerd worden op alle mogelijke problemen. Alle checks worden gecombineerd in 1 conditie met `||` (OF):

```
bash

if [ ! -d "$WERKDIR" ] || [ ! -r "$WERKDIR" ] || [ ! -w "$WERKDIR" ] || [ ! -x "$WERKDIR" ]
    exit 1
fi
```

Uitleg van elke check:

- `! -d` → bestaat niet of is geen map
- `! -r` → geen leesrechten
- `! -w` → geen schrijfrechten
- `! -x` → geen uitvoerrechten op de map

Het script faalt zonder foutboodschap, enkel `exit 1`.

Testen:

```
bash

./huistaascripting.sh ~/bestaatNiet 3 # bestaat niet → exit 1
touch ~/test.txt
./huistaascripting.sh ~/test.txt 3    # bestand, geen map → exit 1
mkdir ~/test_no_read && chmod u-r ~/test_no_read
./huistaascripting.sh ~/test_no_read 3 # geen leesrecht → exit 1
mkdir ~/test_no_write && chmod u-w ~/test_no_write
./huistaascripting.sh ~/test_no_write 3 # geen schrijfrecht → exit 1
mkdir ~/test_no_exec && chmod u-x ~/test_no_exec
./huistaascripting.sh ~/test_no_exec 3 # geen uitvoerrecht → exit 1
```

Claude AI: uitleg req 3+4 — werkdircheck met alle testgevallen

Gesprek 3 — Uitleg requirement 5 (copy_bashrc)

Het script kopieert `~/.bashrc` naar de werkmap en voegt een functie toe die de bash-history wist:

```
bash
cp "$HOME/.bashrc" "$WERKDIR/copy_bashrc"

cat >> "$WERKDIR/copy_bashrc" << 'EOF'

clear_history() {
    history -c
    history -w
}
EOF
```

Uitleg:

- `cp "$HOME/.bashrc"` → kopieert de bashrc van de huidige gebruiker
- `cat >> ... << 'EOF'` → voegt tekst toe aan het einde van het bestand
- `history -c` → wist de history uit het geheugen
- `history -w` → wist de history uit het bestand op schijf

Testen:

```
bash
./huistaascripting.sh ~/werktest 3
cat ~/werktest/copy_bashrc      # toont inhoud met functie onderaan
source ~/werktest/copy_bashrc   # laadt de functie
clear_history                    # roept de functie aan
history                          # toont lege history
```

Claude AI: uitleg req 5 — copy_bashrc en clear_history functie

Gesprek 4 — Uitleg requirement 6 (functie niet dubbel)

De functie mag maar 1 keer toegevoegd worden. We gebruiken een brede regex die ook varianten zoals `function clear_history` of `clear_history ()` herkent:

```
bash
if ! grep -Eq '^[:space:]*(function[:space:]+)?clear_history[:space:]*(\(\))?[
    cat >> "$WERKDIR/copy_bashrc" << 'EOF'
...
EOF
fi
```

Uitleg:

- `grep -Eq` → zoekt met extended regex, stil zonder output
- `^[:space:]*` → mag beginnen met spaties
- `(function[:space:]+)?` → optioneel het woord "function" ervoor
- `clear_history` → de functienaam
- `[:space:]*(\(\))?` → optioneel haakjes erna
- `[:space:]*\{` → gevolgd door een accolade

Testen:

```
bash
./huistaascripting.sh ~/werktest 3
./huistaascripting.sh ~/werktest 3
grep -c "clear_history" ~/werktest/copy_bashrc # moet 1 tonen
```

Gesprek 5 — Uitleg requirement 7 & 8 (tar en gzip)

Alle bestanden in de werkmap worden gebundeld en gecomprimeerd. Het tar-bestand zelf wordt uitgesloten via `--exclude` :

```
bash
tar --exclude='./bestanden.tar.gz' -czf "$WERKDIR/bestanden.tar.gz" -C "$WERKDIR" .
```

Uitleg van elke flag:

- `--exclude='./bestanden.tar.gz'` → sluit het archief zelf uit zodat tar zichzelf niet inpakt
- `-c` → create, maak een nieuw archief
- `-z` → gzip compressie toepassen
- `-f` → geef de naam van het uitvoerbestand op
- `-C "$WERKDIR"` → ga eerst naar de werkmap
- `.` → pak alles in de huidige map in

Testen:

```
bash
./huistaascripting.sh ~/werktest 3
ls -l ~/werktest/bestanden.tar.gz      # bestand bestaat
tar -tzf ~/werktest/bestanden.tar.gz   # toont inhoud
```

Gesprek 6 — Uitleg requirement 9 & 10 (unieke submap)

Bij elke uitvoering wordt een nieuwe unieke submap aangemaakt via datum, tijd en process ID:

```
bash
SUBDIR="$WERKDIR/extracted_$(date +%Y%m%d_%H%M%S)_$$"
mkdir "$SUBDIR"
tar -xzf "$WERKDIR/bestanden.tar.gz" -C "$SUBDIR"
```

Uitleg:

- `date +%Y%m%d_%H%M%S` → geeft datum en tijd, bv `20260413_231037`
- `$$` → het process ID van het huidige script, zorgt voor extra uniekheid
- `mkdir "$SUBDIR"` → maakt de nieuwe unieke map aan
- `tar -xzf` → pakt het archief uit (`-x` = extract)
- `-C "$SUBDIR"` → pak uit in de nieuwe submap

Testen:

```
bash
./huistaascripting.sh ~/werktest 3
sleep 2
./huistaascripting.sh ~/werktest 3
ls ~/werktest/extracted_*      # toont 2 verschillende mappen met inhoud
```

Gesprek 7 — Shellcheck en verbeteringen na review

Na het schrijven van het script werd shellcheck uitgevoerd als linter:

```
bash
shellcheck huistaascripting.sh
```

Eerste run gaf 1 waarschuwing:

- `SC2034: AANTAL appears unused` → variabele `AANTAL="$2"` werd gedeclareerd maar nooit gebruikt. Opgelost door de regel te vervangen door commentaar.

Na de fix:

```
bash
shellcheck huistaascripting.sh && echo "Geen errors gevonden!"
# Output: Geen errors gevonden!
shellcheck huistaascripting.sh
echo $?
# Output: 0
```

Daarna werd het script ook gereviewed via ChatGPT, wat leidde tot deze verbeteringen:

- `echo` bij ongeldige werkdir verwijderd → opdracht vroeg enkel `exit 1`
- Grep regex verbreed → vangt ook `function clear_history` en `clear_history ()` op
- `--exclude='./bestanden.tar.gz'` toegevoegd aan tar → geen waarschuwing meer
- `_$$` toegevoegd aan subdirnaam → extra uniekheid via process ID

Claude AI: shellcheck resultaat en verbeteringen na ChatGPT review